Project Report:Student Stress Detector

Submitted By: Megh Snehalkumar Joshi

Register no.: 25BAI10178

1. Problem Statement

Students often face high levels of academic and personal pressure, yet they frequently lack a simple, private way to track their emotional well-being over time. Without a structured method to log and quantify their moods, it is difficult for students to identify patterns of stress or recognize when they might need to take a break.

2. Why It Matters

Self-reflection is a proven tool for emotional regulation. By transforming subjective feelings into quantifiable data ['Stressed,' 'Neutral,' or 'Positive'] this tool bridges the gap between raw emotion and rational analysis. Providing students with a weekly summary offers a high-level perspective on their well-being, encouraging them to seek support or implement self-care strategies when a pattern of stress is detected. Ultimately, this fosters long-term mental resilience and academic sustainability.

3. Technical Approach & Architecture

The application is built using a modular Python architecture to ensure clean separation of concerns. The system is divided into three primary modules that work in tandem:

- **User Interface (main.py):** Acts as the control hub. It handles the command-line interface, user input, and the orchestrations between the analysis and storage layers. It also utilizes the 'datetime' library to automatically timestamp each entry.
- **Sentiment Analysis Engine (analyzer.py):** Employs the NLTK VADER (Valence Aware Dictionary and sEntiment Reasoner) model. This engine analyzes the linguistic nuance of the user's input to generate a compound sentiment score ranging from -1.0 (highly negative) to +1.0 (highly positive).
- **Data Persistence (storage.py):** Manages the reading and writing of data to a local JSON file ('entries.json'). This ensures that even if the application is closed, the user's history is preserved for future summaries.

4. Key Implementation Decisions

- **Selection of VADER:** Unlike basic word-counting sentiment tools, VADER is specifically tuned for short, informal text. It understands intensifiers (e.g., 'very stressed') and context, making it ideal for personal journal-style entries.

- **Sentiment Thresholding:** I established specific logic to categorize scores: values below -0.3 represent 'Stressed' states, while those between -0.3 and 0.3 are 'Neutral.' This helps simplify complex scores into labels that are immediately actionable for a student.
- **JSON as a Database:** I chose JSON over more complex databases like SQL because it is human-readable, lightweight, and requires zero external configuration for the user, ensuring the project is highly portable.

5. Challenges & Solutions

- **Dependency Portability:** A major challenge was ensuring the NLTK lexicon was available on any machine. I implemented a 'try-except' block or an explicit download call to verify the presence of the VADER lexicon at runtime, preventing the program from crashing on new devices.
- **Error-Resilient File Handling:** To prevent crashes during the first run, the storage module was designed to check for the existence of the data file. If it's missing, the system gracefully initializes a blank history instead of throwing an error.

6. Conclusion & Learning Outcomes

Developing the Student Stress Detector provided a practical education in applying Natural Language Processing to a meaningful social problem. I gained significant experience in modular software design, data persistence strategies, and the importance of creating resilient, portable code that can run on any user's machine without extensive manual setup. This project demonstrates the potential for simple coding solutions to support mental well-being in the academic community.